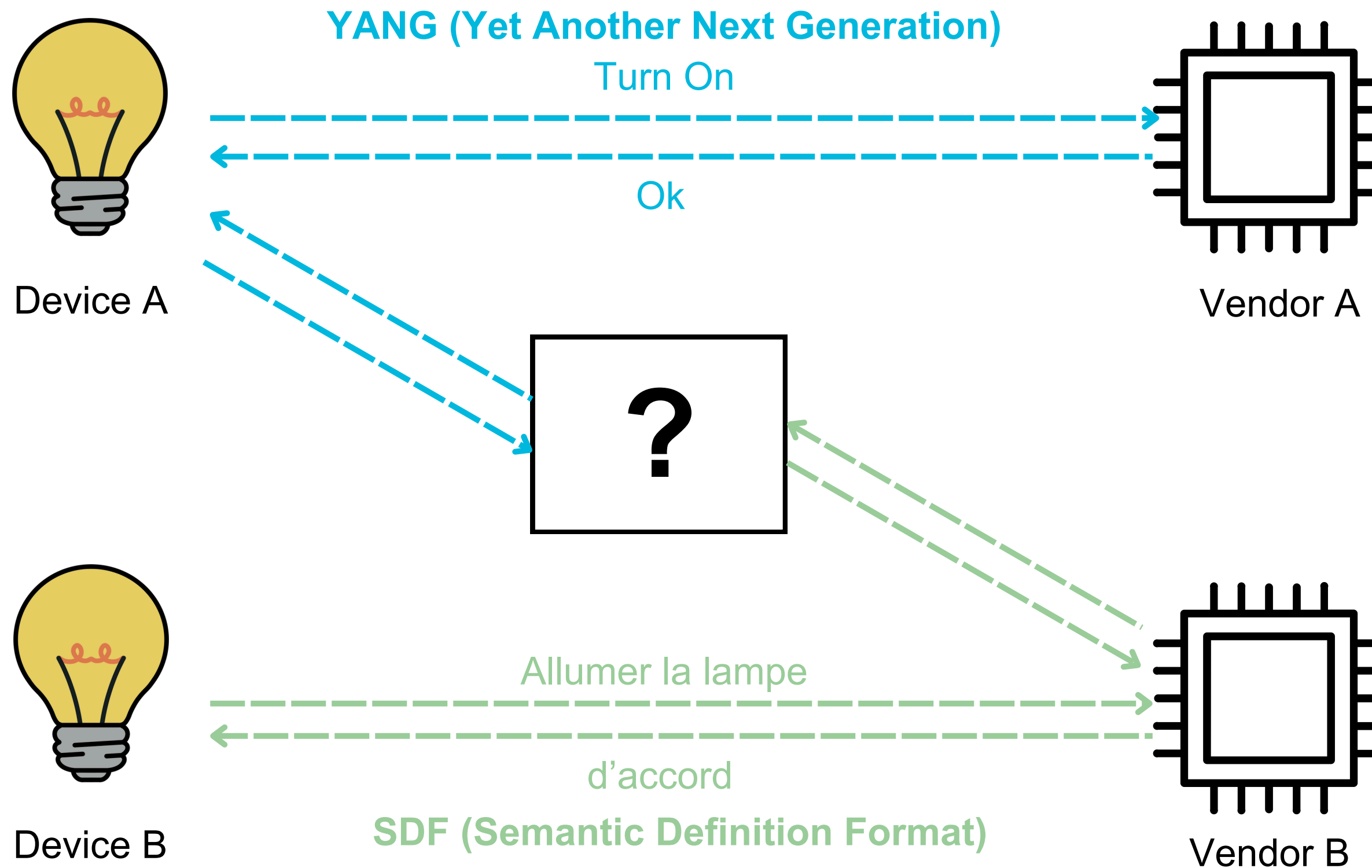


LLM-Assisted Translation Between YANG and IoT Data Representations

Oscar Eduardo López Manchola

Problem



Yet another new generation (YANG)

- Designed for CORECONF.
- Describe networks' configuration.
- Hierarchy-oriented, modular, and recursive
- Based on statement-instance pairs, indentation, and curly brackets.

Semantic Definition Format (SDF)

- Designed to describe objects.
- Based on JSON.
- Lighter and more constrained representation, suited for describing behavior.
- Uses Keyword names to represent object interactions.

Equivalence between SDF and YANG data models

- Semantic equivalence between YANG statements and SDF keyword names fulfilling the same purpose.
- Non-injective equivalence due to differences in the data models structures.

YANG - SDF		SDF-YANG	
module	SDF entity	info	YANG statement
container	sdfObject	sdfObject	container
list	sdfProperty	sdfProperty	leaf
leaf	sdfProperty		leaf-list
	properties		list
notification	sdfEvent	sdfEvent	notificacion
rpc	sdfAction	sdfAction	rpc
action	sdfAction		action

Example conversion

Data models (YANG and SDF)		Data Format (JSON and XML)	
<pre>module sensor_5251{ namespace "http://uri.org/sensor_5251"; prefix sen; leaf sensor_id{ type string; default "TLM0202"; } container measurement{ leaf humidity{ type int8; default 35; } } container capture_time{ leaf date{ type string; default "2026-01-07"; } } }</pre>	<pre>{ "info": { "title": "sensor_5251", "namespace": { "sen": "http://uri.org/sensor_5251", "defaultNamespace": "sen", "sdfProperty": { "sensor_id": { "type": "string", "default": "TLM0202" } }, "sdfObject": { "measurement": { "sdfProperty": { "humidity": { "type": "integer", "default": 35 } } }, "capture_time": { "sdfProperty": { "date": { "type": "string", "default": "2026-01-07" } } } } } } }</pre>	<pre>{ "module": { "name": "sensor_5251", "namespace": "http://uri.org/sensor_5251", "prefix": "sen", "leaf": { "name": "sensor_id", "type": "string", "default": "TLM0202" }, "container": [{ "name": "measurement", "leaf": { "name": "humidity", "type": "float", "default": "35.386485" } }, { "name": "capture_time", "leaf": { "name": "date", "type": "string", "default": "2026-01-07" } }] } }</pre>	<pre><module name="sensor_5251"> <namespace uri="http://uri.org/sensor_5251"/> <prefix value="sen"/> <leaf name="sensor_id"> <type name="string"/> <default value="TLM0202"/> </leaf> <container name="measurement"> <leaf name="humidity"> <type name="float"/> <default value="35.386485"/> </leaf> </container> <container name="capture_time"> <leaf name="date"> <type name="string"/> <default value="2026-01-07"/> </leaf> </container> </module></pre>

Prompt for translation

Place holder

```
module SIDIBELABBESS216 {  
  namespace  
  "http://example.org/SID_216";  
  prefix "SID";  
  container wind {  
    description "desc_Pck_Txt#1";  
    leaf speed {  
      type uint8;  
      units "m/s";  
      default 15;  
    }  
  }  
}
```

Instruction

1. Transform the key-value pair; the key will change, but the value remains.
2. Keep the same number of description fields and elements within the hierarchy.
3. Maintain an even number of opened and closed curly brackets.
4. Keep the numeric values during the translation."

Alpaca format + Harness

Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

Instruction:

<transformation_rules>

{instructions}

</transformation_rules>

<output_contract>

Start the response with the model's first structural character.

End the response with the final closing structural character.

After the final closing character, emit the EOS token and stop — generation is complete.

Produce plain text only; no reasoning, no verification text, no repetition.

</output_contract>

Input:

{data_representation}

Response:

{data_translated}

Prompt for knowlegde

Knowlegde

RFC 7950 (YANG) -> The "leaf" statement is used to define a leaf node in the schema tree. It takes one argument, which is an identifier, followed by a block of substatements that holds detailed leaf information.

RFC 9880 (SDF) -> sdfProperty is used to model elements of state within Things modeled by the enclosing grouping. A named definition entry in an sdfProperty may be associated with some protocol affordance to enable the application to obtain the state variable and, optionally, modify the state variable.

Alpaca format

###Instruction

You will receive knowledge concerning the semantic equivalence of Yet Another Language Format (YANG) data models and Semantic Definition Format (SDF). Use the context to choose the correct mapping

###Input

Which is the equivalent model construct of a leaf in YANG at top level or one level below?

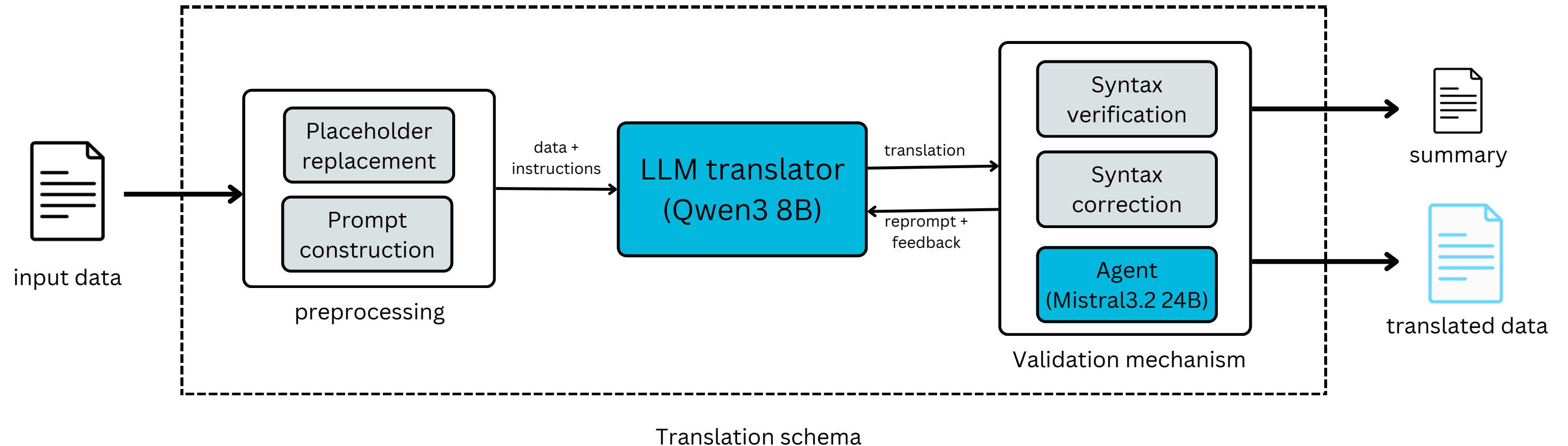
###Output

The equivalent model construct is sdfProperty with a simple type such as integer, number, boolean, or string.

Database for fine-tuning

Knowlegde	Data representation examples	
	Schema generated	Vendors repositories
 	<pre>module sensor{ namespace "http://uri.org/sensor"; prefix sen; container timestamp{ leaf hour{ type string; default sensor[hour]; } } leaf sensor_id{ type uint8; default sensor[sensor_id]; } leaf measure{ type uint8; default sensor[measure]; } }</pre>	  

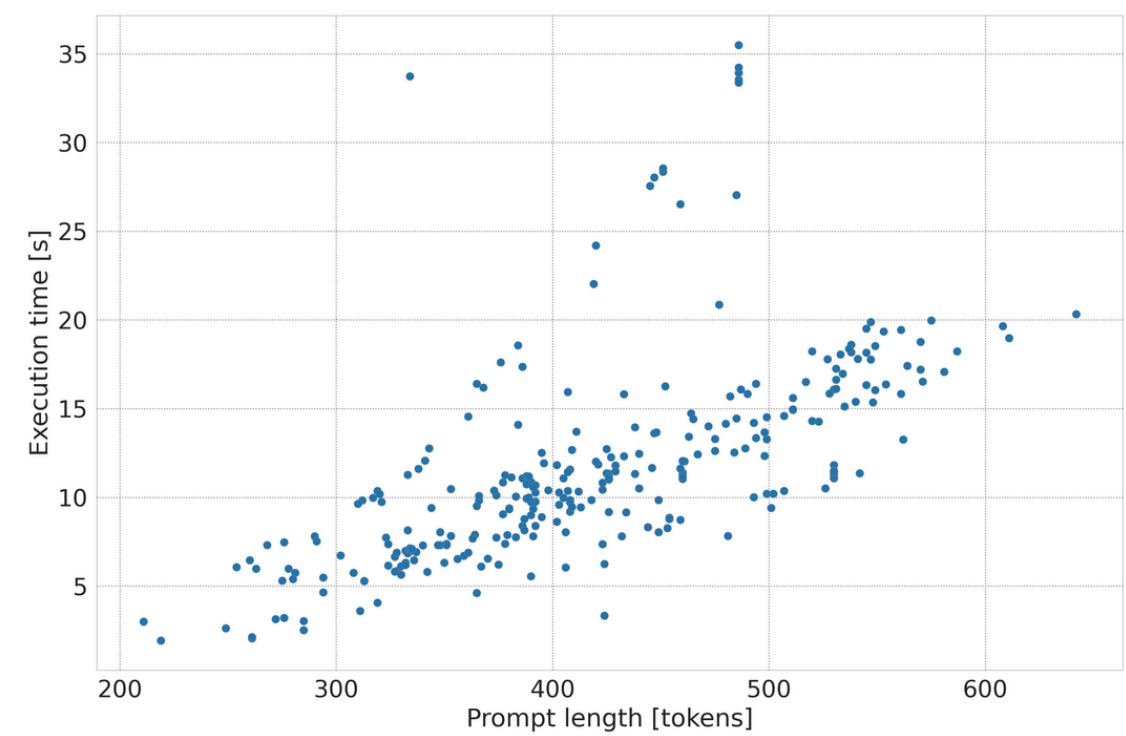
Translation schema Based on LLM



- Deterministic functions paired with LLM's probabilistic output.
- Deterministic syntax evaluation and the agent judgment for semantics.
- YANG as the preferred representation.

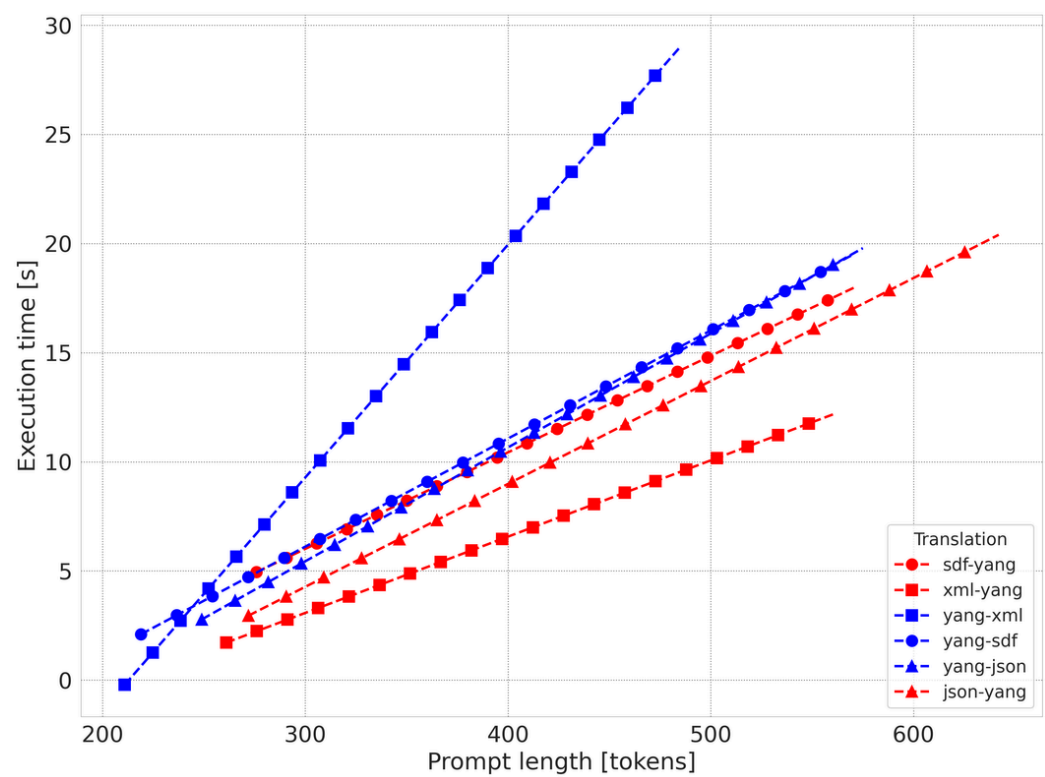
Framework Performance

Execution time



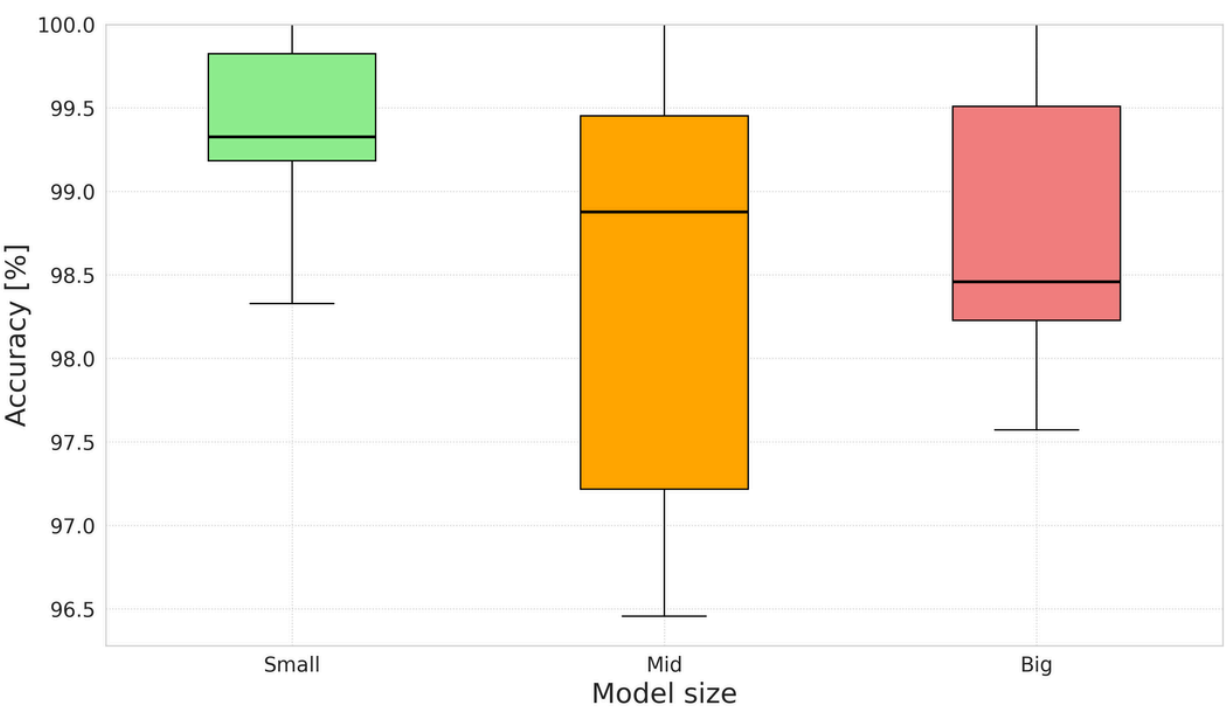
- Slow performance as the prompt length increases
- linear dependence from prompt length.

Source and target representation



- Fast from any representation to YANG.
- Slow from YANG to any representation.
- linear dependence from prompt length.

Accuracy



- Performance decreases with larger prompts.

Framework Performance

Accuracy base model

Representations	Result	Translation	Porportions
YANG/SDF	Success	13	11.31%
	Error	102	88.69%
YANG/JSON	Success	45	40.17%
	Error	67	59.82%
YANG/XML	Success	7	11.11%
	Error	56	88.89%

- Improved performance after fine-tuning
- Lower accuracy with XML-based data representations.
- Syntax error near 0 for YANG and JSON-like translations.

Accuracy fine-tuned model

Representations	Result	Translation	Porportions
YANG/SDF	Success	111	96.52%
	Syntax error	4	3.48%
	Semantic error	0	0%
YANG/JSON	Success	112	100%
	Syntax error	0	0%
	Semantic error	0	0%
YANG/XML	Success	51	69.86%
	Syntax error	17	23.28%
	Semantic error	3	6.85%

Framework Performance

Accuracy roundtrip translation

Representations	Result	Translation	Porportions
YANG/SDF	Success	109	96.52%
	Syntax error	3	3.48%
	Semantic error	3	0%
YANG/JSON	Success	104	100%
	Syntax error	7	0%
	Semantic error	1	0%
YANG/XML	Success	39	69.86%
	Syntax error	30	23.28%
	Semantic error	4	6.85%

Roundtrip translation

Representations	Result	Translation	Porportions
YANG	Success	76	48.71%
	just semantic	31	19.87%
	No semantic	49	31.41%
SDF	Success	37	67.27%
	just semantic	8	14.54%
	No semantic	10	18.18%
JSON	Success	15	27.27%
	just semantic	22	40%
	No semantic	18	32.72%
XML	Success	9	26,47%
	just semantic	1	2,94%
	No semantic	24	70.58%

Limitations

- LLM performance
 - Hallucination of both translator and agent
- Validation layer.
 - Lack of robust deterministic methods to validate the data structure integrity.
 - Evaluate performance on different LLM models.

Expected work plan

- Work on the harness (LLM environment)
 - Evaluate the harness methodology and its effects.
 - Implement grammar rules (BNF,...) to constrain the LLM output.
 - Integrate retrieval-augmented generation (RAG).
- Work on the LLM.
 - Evaluate performance on different LLM models (translator and agent).
- Implement robust data structure comparison.
- Apply the framework to an embedded environment.
 - Testbed deployment.
- Expand the translation task to context-intent extractions.



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom



**LLM-Assisted Translation Between
YANG and IoT Data Representations**

Oscar Eduardo López Manchola
T2TRG2 20.07.2025

THANK YOU

Questions?